

AgroIQ Reference Architecture: Technical Record

Carlos Andrés Yoncón Changkuón

Version 12, 1 October 2026

Cite as: Yoncón Changkuón, C. A. (2026). *AgroIQ Reference Architecture: Technical Record*, version 12. <https://doi.org/10.5281/zenodo.23087515>. Released under CC BY 4.0.

Scope

This is an engineering record of a low cost architecture for soil condition sensing and irrigation decision support, designed and built by one person over seven years. It sets out the architecture as built, the design decisions behind it, the alternatives tested and discarded, and the components that remain incomplete or unvalidated.

It is written for engineers, extension staff and growers who might build this architecture, evaluate it, or take parts of it for their own work, and it is published so that they can.

What it documents is the architecture: the node, the radio layer, the division of work between the network server, the storage services and the application, and the security controls each layer needs. The source code and the configuration of the development instance are not published. Section 5.11 sets out the requirements any deployment of the architecture has to meet, whoever builds it.

It documents personal engineering work. Customers, revenue and commercial terms are outside its scope. Where section 8 cites prices published by other parties, it does so as a benchmark for a design constraint. Section 9 is the register of everything the architecture still requires.

Implementation status is stated for every component, and it refers to the development instance. The document distinguishes five conditions. They are applied in the headings of sections 5 and 7, where implementation status is the point at issue.

Label	Meaning
Implemented	Built, deployed and in operation
Partially implemented	Deployed with a stated deficiency
Available, not configured	Supported by the current stack, not enabled
Not implemented	Designed or intended, no working code
Not validated	Present in the system, without evidence supporting its correctness

Limitations appear inline, labeled **Limitation**, in the subsection to which they belong. Section 9 consolidates them into a single register with cross references.

Scope of publication and license

This document is the reference design for the AgroIQ architecture, and it is published in full.

The architecture described here, the bill of materials in section 8, the design decisions and rejected alternatives recorded throughout, the register of open items in section 9, the payload specification in Appendix A, and the provisioning and calibration procedures are released for anyone to build, adapt and deploy, under the Creative Commons Attribution 4.0 International License (CC BY 4.0). Attribution is the only condition. Appendix A gives the frame format in enough detail for an independent builder to write a decoder in any language. My own decoder is a JavaScript function tied to one network server and would be of little use to anyone else. The current text of this notice, with its publication date, is maintained at agroiq.metrिकास.net/license. This version of the record is archived at <https://doi.org/10.5281/zenodo.23087515> and the working copy is at <https://github.com/cyoncon/agroiq-reference-architecture>.

The development instance that runs this architecture, with its source code, configuration, credentials and data, is not part of this publication. Metrics.net LLC holds the equipment, cloud accounts and site agreements for that work, and each site agreement leaves the farm's data with the farm, as section 5.6 records.

The separation answers one of the barriers this design is built against. The Government Accountability Office identified three reasons precision agriculture has not reached small and midsize farms: what the equipment costs to acquire, uncertainty over who owns the data and who may share it, and the absence of uniform interoperability standards. An architecture only its author can build adds to the last of those, which is why the design is published. The second is settled in writing with each farm before a sensor goes into the ground.

1. Design constraints

Four constraints governed the design.

Unit cost had to remain within reach of a small or mid size operation. This ruled out any recurring per node connectivity charge, ruled out research and laboratory grade instrumentation at several thousand dollars a station, and favored an integrated development board over discrete modules. It did not rule out industrial grade sensing, and section 3.1 records why an industrial probe was chosen over a cheaper one.

Field connectivity is limited or absent. The system has to operate where there is no cellular signal at the node and no practical route for cable. This determined the communication layer.

Nodes are unattended. Power has to sustain itself, and the system has to recover from transient failures without an on site visit.

The destination sector is designated critical infrastructure. The Department of Agriculture and the Department of Health and Human Services are the co Sector Risk Management Agencies for the Food and Agriculture Sector, and the Department of Homeland Security's 2018 report Threats to Precision Agriculture found that these tools widen the sector's

attack surface and that many connected devices carry weak security. An architecture built for wide deployment in this sector has to be engineered to its security requirements from the start. This constraint governs the communication layer, the tenancy model and the credential handling, and section 5 records the resulting control set component by component, including the controls that are not yet in place.

1.1 Exposed surface compared with the alternatives

A node in this architecture has no internet address and runs no service that accepts connections. Its firmware switches off the ESP32's Wi-Fi and Bluetooth radios at start. It transmits to a gateway on an unlicensed band, with the payload encrypted and every frame authenticated under keys unique to that device, as section 5.1 records. It listens only in the two short receive windows that follow its own transmissions, for downlinks that reach it through the network server. The devices on a farm with an internet connection are therefore its gateways, one for most small farms, and their number does not rise as nodes are added.

The two alternatives considered in sections 4.4 and 8.3 distribute that differently. A cellular connected sensor carries its own modem, its own subscription and its own path to a carrier network, so a deployment of twenty sensors is twenty independently connected devices to provision, monitor, credential and retire, rather than one. A platform that offers no self hosting option carries the operator's data and its only telemetry path on infrastructure the operator does not control, so segmentation, monitoring and retention of that path are not decisions the operator can make. Neither observation is a criticism of a product. Both describe topology, and a topology gets chosen once, at the time a segment connects.

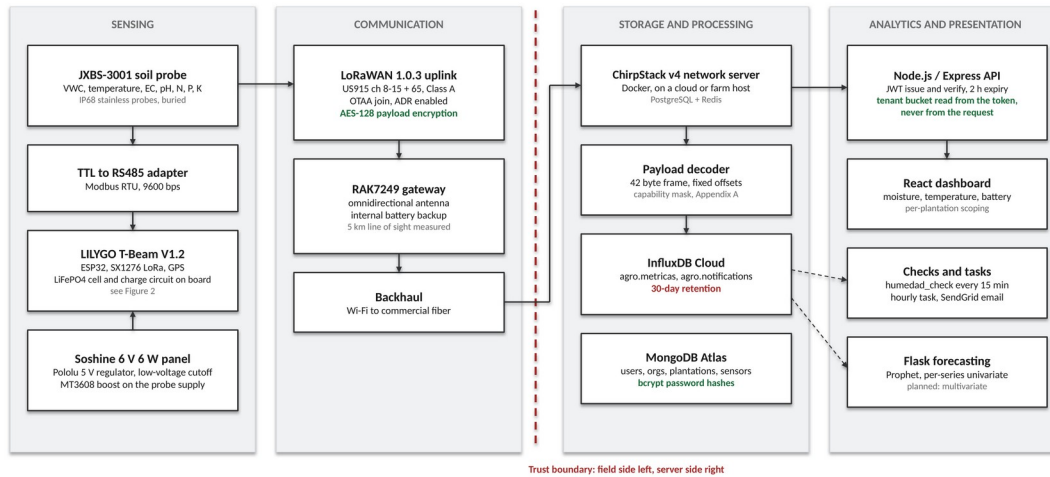
Limitation. Low exposed surface is a property of the topology. It says nothing about how well a particular deployment is hardened, which depends on the controls in section 5 and the deployment requirements in section 5.11. Of those controls, gateway authentication, encryption of the internal transports and tenant separation at the network server are not yet in place, and the register in section 9 carries them.

2. System architecture

The system is organized in four layers. Figure 1 shows the complete path from probe to dashboard.

Figure 1. AgrolQ system architecture and data path

Field acquisition through to dashboard. The dashed line marks the boundary between the equipment in the field and the server side.



Payload is encrypted at the node under session keys derived from a per-device AppKey, and decrypted at the network server. The API never receives a bucket name from the client: it is read from the verified token. Open items are detailed in the register: gateway identified by EUI only, no reading integrity checks, no alert when a node goes silent.

Data path: the probe communicates over RS485 to a TTL to RS485 adapter, which converts to TTL for the ESP32 on a LILYGO T-Beam board. The probe's 5 V supply is raised from the LiFePO4 cell by an MT3608 boost converter on the sensor feed. The ESP32 assembles a LoRaWAN uplink, transmitted on US915 to a RAK7249 gateway. The gateway backhauls over Wi-Fi to a fiber connection and forwards to the ChirpStack v4 network server. ChirpStack decrypts the frame, applies a JavaScript decoder, and writes the decoded fields into InfluxDB Cloud. A Node.js API reads from InfluxDB and serves the React dashboard. MongoDB Atlas holds identity and tenancy. A separate Flask service performs forecasting.

The network server and the application run in Docker containers. In the development instance they run on a workstation, reached from outside through Cloudflare Tunnel, so that no port other than the gateway's is open to a network; InfluxDB Cloud and MongoDB Atlas are the storage layers. The architecture does not depend on where the containers run. A small cloud instance with a fixed public address, or a host on the farm itself, satisfies it equally, and section 6.5 sets out what each costs.

3. Sensing layer

3.1 Probe selection

The deployed instrument is the **JXBS-3001 7 in 1 soil composite sensor** from Weihai Jingxun Changtong. It reads volumetric water content, temperature, electrical conductivity, pH, and nitrogen, phosphorus and potassium from a single body.

Parameter	Range	Stated accuracy	Resolution
Temperature	-40 to 80 °C	±0.5 °C	0.1 °C
Moisture (VWC)	0 to 100 %	±3 % below 53 %,	

Parameter	Range	Stated accuracy	Resolution
		$\pm 5\%$ above	
Electrical conductivity	0 to 10,000 $\mu\text{S}/\text{cm}$		10 $\mu\text{S}/\text{cm}$
pH	3 to 9	± 0.3 pH	
Nitrogen	0 to 1999 mg/kg	$\pm 2\%$ F.S.	1 mg/kg
Phosphorus	0 to 1999 mg/kg	$\pm 2\%$ F.S.	1 mg/kg
Potassium	0 to 1999 mg/kg	$\pm 2\%$ F.S.	1 mg/kg

It operates on 5 to 24 V DC, draws under 0.15 W, communicates over Modbus RTU on RS485 at 9600 bps, and is rated IP68 with stainless steel probes in an epoxy sealed body.

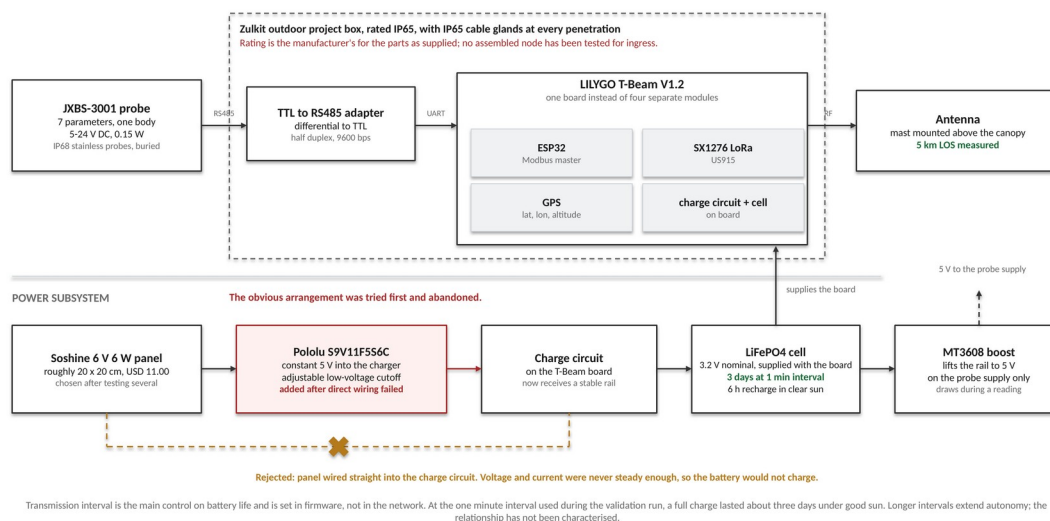
Selection criterion: service life. The first builds used capacitive moisture and temperature sensors costing approximately five dollars per unit. These corroded at the soil contact points within two to three weeks. At an unattended node, a component with that service life is a recurring replacement cost rather than a fixed one, and it removes the cost advantage that motivated the choice. The RS485 interface proved to be a second advantage: Modbus is an industrial standard and interfaces to an ESP32 through a TTL to RS485 adapter without protocol level development.

3.2 Power subsystem

Figure 2 shows the node internals and the power path.

Figure 2. Sensor node integration and power subsystem

Signal path across the top, power path across the bottom. The regulator between panel and charge circuit is what made the design work.



Each node runs on the LiFePO₄ cell that ships with the T-Beam board, charged by a Soshine 6 V 6 W panel of approximately 20 by 20 cm at a cost of eleven dollars. At a one minute transmission interval, a full charge sustains operation for approximately three days, and recharges in about six hours of clear sun.

The transmission interval is the principal control on battery life and it is set in the device firmware, not in the network. A one minute interval is far finer than an irrigation decision requires, because soil moisture is a slowly varying quantity; it was used during development for visibility into the system. Lengthening the interval reduces the number of radio transmissions per day, and it can be changed without any hardware change. Whether that translates into a specific autonomy figure has not been measured, and the limitation below records it.

The charging arrangement departs from the reference design for the board.

The first build wired the panel directly into the T-Beam onboard charging circuit, which is the intended configuration for that board. The battery did not charge reliably. The cause was supply instability at the charge controller: panel output varies with cloud cover and sun angle, and the controller requires a steadier input than the panel provides.

A **Pololu S9V11F5S6C step up/step down regulator** was inserted between the panel and the board, holding a constant 5 V into the charging circuit. This component is what produced a predictable autonomy figure; before it, autonomy was not predictable at any interval.

The regulator was chosen for a second property as well: an adjustable low voltage cutoff, which disables the output when the input falls below a set threshold instead of letting the supply sag. A board browning out under a failing supply can write corrupt state, or hang somewhere a watchdog does not catch it. A clean cutoff gives a node that is simply off until the panel recovers, and a node that is off is easier to design around than one that is half working.

The probe is a 5 V DC instrument and the cell is LiFePO₄ at a nominal 3.2 V, so the probe cannot be driven from the cell directly. An **MT3608 boost converter** sits on the probe supply and raises the rail to 5 V. It is on the sensor feed rather than on the board supply, so it draws only while a reading is being taken.

Panel selection was determined empirically. Several generic and branded retail panels were tested. Most did not deliver output close to their rated figure. The Soshine unit produced usable power under limited sun exposure within the size and cost envelope, and was adopted as the standard part.

Limitation. The relationship between transmission interval and autonomy has not been characterized empirically. The three day figure is measured at a one minute interval. Longer intervals are expected to extend autonomy, and the extent has not been measured. Performance under extended overcast is likewise not characterized.

3.3 Enclosure and mounting

The node is housed in a Zulkait outdoor project box rated **IP65**, with IP65 cable glands at every penetration, one for the probe lead and one for the panel lead. The assembly is mast mounted, which lifts the node above the canopy for radio line of sight and keeps the enclosure clear of standing water and soil contact.

The first builds, before the RAK gateway was adopted in 2022, used an unrated plastic container with a gasketed lid. Ingress of moisture, dust and insects into the electronics compartment was a real failure mode in that configuration, and it is what drove the change to a rated housing. IP65 is dust tight and protected against water jets from any direction, which

covers the exposure a mast mounted node sees. The probe is separately rated IP68 and is buried.

Limitation. The IP65 figure is the manufacturer’s rating for the enclosure and the glands as supplied. I have not run an independent ingress test on an assembled node with its penetrations made, and no node has yet completed a full growing season in the field.

3.4 Calibration

Temperature and moisture were calibrated once, against a reference thermometer and hygrometer, immediately after firmware was flashed to the ESP32. No recalibration has been performed since. Readings have remained consistent, indicating that drift on these two parameters is not a practical concern at this timescale.

Limitation. The basis for that statement is a single calibration event, supported by observed stability over time rather than by repeated comparison against a reference. No calibration schedule with periodic reference checks exists.

NPK is not calibrated and is not usable; section 9 records the finding and the evidence for it.

4. Communication layer

4.1 Protocol and configuration

LoRaWAN 1.0.3, Class A, regional parameters revision A, region US915 on channels 8 to 15 plus 65.

Setting	Value
Network server	ChirpStack v4.19.2
Activation	OTAA
MAC version	LoRaWAN 1.0.3
Device class	A
ADR	Enabled, default LoRa only algorithm
Expected uplink interval configured in device profile	3600 s (network server inactivity threshold, not the transmission period)
Transmission interval	Set in device firmware. On the deployed nodes the network server records an uplink about once a minute, a cycle that includes powering the probe and waiting for a GPS fix. The interval triples at night and when the battery is low, and a downlink command switches the node to a faster mode
RX1 delay	1 s
RX2 data rate and frequency	DR8, 923.3 MHz

The expected uplink interval in the device profile is a monitoring parameter. ChirpStack uses it to decide when a device has stopped reporting and should be flagged as inactive; it does not govern how often the node transmits. The transmission period is set in the ESP32 firmware and is independent of it.

Spreading factor is not pinned. Adaptive data rate is enabled, so the network server adjusts it per node on observed link quality, trading range against airtime. Uplink channels are the standard US915 sub band: 903.9 through 905.3 MHz at DR0 to DR3, plus 904.6 MHz at DR4. The 42 byte frame of Appendix A exceeds the 11 byte maximum payload at DR0, so it can be sent only at DR1 or above; the deployed nodes were observed at DR1.

4.2 Payload decoding

Decoding is performed by a JavaScript function configured in the ChirpStack device profile. It unpacks the binary uplink into latitude, longitude, altitude, HDOP, satellite count, battery voltage, soil temperature, soil moisture, EC, pH, and N, P and K.

Byte 17 of the frame carries a capability mask. The mask does not change where fields sit: every field is read at a fixed offset and the frame is of fixed length. It records which sensors were present and produced a reading on that uplink, so the server decodes only the fields the node vouches for and leaves the rest out of the decoded record.

Appendix A publishes the frame format in full, with every offset, width, byte order, scaling factor and mask bit, so that an independent implementer can write a decoder without access to mine.

Measurement keys registered in the profile: K, N, P, altitude, capab, ec, hdop, latitude, longitude, ph, rain, sats, soil_moist (typed Gauge, labeled Soil Moisture), soil_temp (typed Gauge, labeled Soil Temperature), voltage.

4.3 Gateway

A **RAK7249** outdoor gateway with an omnidirectional antenna and an internal battery for power redundancy. Backhaul is Wi-Fi to a commercial fiber service.

Measured range is 5 km line of sight, without obstruction, from gateway antenna to a T-Beam node.

The current deployment has one gateway online and four registered nodes: SOIL-04-A1, SOIL-06-A1, SOIL-07-A2 and TBEAM-SPH05. Six nodes exist in total, as recorded in section 8.1.

Limitation. The practical node ceiling per gateway has not been characterized.

4.4 Selection of LoRaWAN

Of the design decisions in this document, this is the one that most directly determines whether a per device recurring charge exists at all.

Cellular imposes a recurring subscription on every node, which is the cost structure section 1 rules out, and rural field coverage cannot be assumed. **Wi-Fi** was rejected on range: covering a field at the distances involved requires repeaters, with signal degradation across them, against

a LoRaWAN link measured at 5 km from a single gateway. **Satellite** is technically workable and was rejected on cost per node. These are design judgments about fit against the constraints in section 1. No coverage survey or per node price comparison was carried out.

LoRaWAN concentrates infrastructure cost into a single shared gateway. The node side carries no recurring connectivity charge.

Cellular and satellite were rejected for the nodes, not for the whole system. Both remain options for the **gateway backhaul** at sites without wired internet. The resulting pattern is: nodes join the gateway over LoRa, and the gateway reaches the internet over whatever transport is available at the site.

An earlier gateway built on a Raspberry Pi with a LoRa module served for initial testing. Its antenna output was insufficient and it did not approach the 5 km link achieved by the RAK7249, so the RAK unit took over as the deployment gateway in 2022. The Raspberry Pi gateway continued to run alongside it and was disconnected during 2023. Its configuration remains in ChirpStack with a last seen date of June 2023.

4.5 Node deregistration and reset behavior

Uplinks are unconfirmed, so a lost frame is not retransmitted, and packet loss has not been quantified. A separate failure mode was observed: nodes would intermittently lose registration with the network server and stop reporting without reporting a fault. The failure was silent, with no alarm and no other symptom.

The measure adopted is a **node initiated reset after every fifty uplinks**, about fifty minutes at the daytime interval, which forces a new join and restores reporting. The suspected cause is timing drift in the node over a long session; it has not been confirmed.

Limitation. The reset addresses the symptom rather than the cause. It does not stand in the way of frame counter validation, because every reset ends in a new join, as section 5.2 records.

5. Security and data governance

Each control below carries its implementation status. Planned work is consolidated in section 5.9.

5.1 Device authentication and payload encryption. Implemented.

Devices join by **OTAA** using a per device Application Key provisioned at registration alongside the DevEUI. On join, the network server derives the session keys, the Network Session Key and the Application Session Key, and admits the device only if the join request authenticates against the provisioned AppKey.

This provides AES-128 encryption of the payload and a message integrity code on every frame, native to LoRaWAN rather than added above it. It also prevents substitution or injection of a node without the corresponding key and identifier pair.

OTAA was selected over ABP. ABP writes session keys statically into the device and reuses them indefinitely, so an exposed session key stays valid and rotation requires reprovisioning. OTAA derives fresh session keys on every join.

5.2 Frame counter validation. Implemented.

Frame counter validation is **enabled** for the deployed devices. The network server rejects any frame whose counter is not higher than the last one observed, so a captured frame cannot be replayed.

The firmware keeps its frame counters in RTC memory across software restarts and deep sleep, and the periodic reset described in section 4.5 ends in a new join, which restarts the counters on both sides. Validation and the reset therefore run together.

Until 30 September 2026 validation was switched off at the network server, on the belief that the reset desynchronized the counter. A review of the firmware showed that it does not, and validation was enabled and tested that day.

5.3 Gateway authentication. Available, not configured.

The gateway reaches the network server over the Semtech UDP packet forwarder protocol, on port 1700. That protocol carries no authentication: the server identifies the gateway by its EUI alone, and any host that can send UDP to that port can present itself as a gateway. ChirpStack also accepts gateways over LoRa Basics Station, which authenticates the gateway with a client certificate over TLS. The deployment already runs the Basics Station bridge, and the RAK7249 supports the protocol.

Limitation. The gateway has not been moved to Basics Station and no certificate has been issued, so it is identified by EUI alone. In the development instance, port 1700 is reachable only from the local network, because no port is forwarded to it from the internet. Listed in section 5.9.

5.4 Application authentication. Implemented.

Users authenticate against MongoDB Atlas. Passwords are stored as bcrypt hashes with a per user salt, never in plaintext.

On successful login the API issues a JSON Web Token with a two hour expiry, signed with a server side secret. The token payload carries the user identifier, the organization identifier, and the InfluxDB bucket names belonging to that organization.

Section 5.11 sets out the requirements for account creation, token verification and rate limiting that any deployment of the application layer has to meet.

5.5 Tenant isolation. Partially implemented.

The tenant data scope is taken from the signed token and never from the request.

When the API handles a query, the InfluxDB bucket it reads from is read from the verified JWT, not from the query string. Request parameters are validated for presence and shape before a Flux query is constructed, and a request missing required parameters is rejected.

Consequently a user cannot reach another organization's data by modifying a request, because the request does not name the store it reads from. Tenant separation is applied at the server, on every query, and does not depend on client behavior.

Above that layer, the data model scopes hierarchically: organization, then plantation, then sensors. An administrative user sees every plantation in their farm. A standard user sees only the plantations associated with them.

Limitation. There is no role system beyond this hierarchy, and the network server layer operates with a single administrative account without separation of duties.

5.6 Data ownership

Who owns the data is settled in writing with each site before a sensor goes into the ground. The executed field validation agreement leaves ownership of the farm's agronomic and operational information with the farm, and permits the operator to use that data in de identified or aggregated form.

The architecture supports what a site agreement needs to promise beyond that. Each farm's readings sit in an InfluxDB bucket of their own, the scope that section 5.5 enforces on every query (the development instance serves one site and holds one), so they can be exported in an open format and deleted as a unit when the farm asks or when the work ends.

5.7 Reading integrity. Not implemented.

There is **no mechanism to detect a manipulated reading or a compromised node** beyond LoRaWAN cryptographic authentication of the frame. A physically compromised node transmitting authenticated but incorrect values is not detected downstream.

The intended controls, none of them built:

Range validation against each sensor published measurement envelope, so that physically impossible values are flagged or rejected rather than stored.

Rate of change validation, flagging transitions between consecutive readings larger than soil physics permits over the sampling interval. This covers sensor failure and injected values with a single check.

Silence detection, raising an alert when a node stops reporting for longer than a set multiple of its uplink interval. No equivalent exists today, so a node can go dark without notification until someone inspects the dashboard or the network server, as happened in September 2026 (section 11).

5.8 Backup and recovery. Partially implemented.

Backups are in place: manual export from MongoDB Atlas, the InfluxDB Cloud backup facility, and container snapshots of the ChirpStack deployment.

Limitation. No recovery protocol is documented or tested. Recovery time currently rests on the availability guarantees of the managed services. Design and testing of that procedure is open.

5.9 Planned security work

The following are supported by the current stack but not enabled or not built. A capability the stack supports is not the same as one that is switched on, and the section cited against each item gives its status.

Gateway authentication, by moving the gateway to Basics Station with a client certificate, as in section 5.3.

Silence detection, an alert when a node stops reporting for longer than a set multiple of its uplink interval, as described in section 5.7.

Per farm tenants in ChirpStack. All devices currently sit under a single tenant. ChirpStack supports separate tenants, which would isolate device registries and API access per farm rather than relying on downstream isolation alone.

Private gateways. The uplink and downlink private flags are both off, so the gateway is not restricted to its own tenant. Enabling them would prevent devices of another tenant from using it.

MQTT over TLS on the gateway bridge, and TLS on the PostgreSQL and Redis connections inside the ChirpStack deployment; section 5.10 describes the last two.

Scoped InfluxDB tokens per tenant. The API authenticates to InfluxDB with tokens scoped to the service rather than to the tenant: one for the data path and a second for reading the check and task definitions. Neither is per tenant, so a compromised API process could read beyond the tenant it is serving. Per tenant scoped tokens would close that.

Shorter JWT expiry with refresh tokens, and audit logging of access.

LoRaWAN 1.1. The deployment runs 1.0.3. Version 1.1 separates the network and application root keys and protects the join procedure against replay with a counter based DevNonce. Migration requires device firmware changes and is a larger piece of work.

5.10 Internal transport encryption. Available, not configured.

The ChirpStack deployment runs its PostgreSQL connection with `sslmode=disable` and its Redis connection with TLS off. Both are internal to the container network and not externally exposed.

Limitation. Both connections are unencrypted in transit within that boundary. Remediation is listed in section 5.9.

5.11 Deployment requirements

How an instance is deployed decides as much of its security as the design does. The requirements below apply to any deployment of this architecture. They come from reviewing the development instance against the reference configurations of its components, and each one answers a weakness that configuration alone can create.

Replace the network server's API secret. ChirpStack ships its reference configuration with a placeholder secret, and tokens signed with that value are valid for anyone who has read the same file.

Keep the MQTT broker off every network interface. The gateway bridge and the network server reach it inside the container network, and nothing else needs to.

Put the network server console and any REST interface behind authentication and TLS, or bind them to the host's loopback address and reach them through an encrypted tunnel.

Keep credentials out of container images and repositories. Hold them in files on the host readable only by the service that uses them, and exclude environment files explicitly from every build context.

Terminate TLS in front of the application, so that passwords and tokens never cross a network in the clear.

Restrict database access to the addresses of the application hosts.

Create user accounts administratively, with their organization and data scope assigned at creation, and expose no public registration.

Make token verification fail closed, so that a request whose token fails verification ends there.

Rate limit the authentication endpoints, and accept cross origin requests only from the application's own origin.

Keep every service on a runtime version that still receives security fixes.

Open only the port the gateway needs, and restrict its source to the gateway's address where that address is fixed.

The development instance was reviewed against these requirements on 29 September 2026 and brought into line with them, with four exceptions that the register in section 9 carries. The forecasting service runs on Python 3.9, which no longer receives security fixes. The gateway port accepts traffic from the whole local network rather than from the gateway's address alone. InfluxDB Cloud is reached by token alone, because the plan in use offers no address restriction. And the environment files that hold the credentials are readable by the workstation's user account, not only by the services that use them.

5.12 Assessment against NIST IR 8259A

NIST IR 8259A, the IoT Device Cybersecurity Capability Core Baseline published by the National Institute of Standards and Technology in May 2020, sets out six capabilities a connected device should support. The node is assessed against each of them below. Sections 5.1 to 5.10 cover the network server and the application, and section 5.11 the requirements for deploying them.

Capability	Status	How the node meets it, and what is missing
Device identification	Met	Each node carries a unique

Capability	Status	How the node meets it, and what is missing
		DevEUI and its own AppKey, and the network server admits it only through an authenticated OTAA join (section 5.1). The identity is not bound to tamper resistant hardware, which the data protection row covers.
Device configuration	Partially met	The transmission interval and the sensor mask are set in firmware and change by reflashing over USB, which requires physical access. The node also accepts five commands by downlink: switch a relay output on or off, restart, and enter or leave a faster transmission mode. Downlinks are encrypted and authenticated under the device's session keys, so only the network server can issue them. The deployed nodes run firmware version 30, which copies a downlink without checking its length; version 31 adds that check and has not yet been loaded onto them. There is no record of configuration changes on the node.
Data protection	Partially met	In transit, the payload is encrypted and every frame is authenticated with keys unique to the device. On the node, the AppKey from which those keys derive is compiled into flash without encryption, so physical access to a node exposes it. Replay is prevented by frame counter validation at the

Capability	Status	How the node meets it, and what is missing
Logical access to interfaces	Met	network server (section 5.2). The node accepts no connection over the radio and listens only in the receive windows after its own uplinks. Its Wi-Fi and Bluetooth radios are switched off in firmware at start. Its USB serial port can be reached only by opening the IP65 enclosure.
Software update	Partially met	Firmware is updated over USB with physical access. There is no remote update path to attack, and there is also no signed update and no secure boot.
Cybersecurity state awareness	Partially met	The network server records each node's joins and its last uplink, and nothing yet raises an alert when a node goes silent (section 5.9). The node does not detect or report tampering.

Two capabilities are met and four are partially met. What each one is missing is carried in the register at section 9.

6. Storage and data handling

6.1 Time series store

InfluxDB Cloud, on AWS, region us-east-1.

Bucket	Retention	Purpose
agro.metrics	30 days	Sensor readings
agro.notifications	30 days	Alert records
_monitoring	7 days	Check statuses
_tasks		Task execution records

Schema type is implicit. Measurements are written one per decoded field as `device_frmpayload_data_<field>`, plus `device_uplink` for link metadata. Series carry

tags for device name, application and DevEUI, which is what enables per node and per plantation filtering downstream.

Limitation. Retention is bounded at thirty days by the current service tier. The platform therefore does not hold a season length history, and any analysis requiring multi season data would need either a retention change or an export and archive step. Neither exists.

6.2 Identity and configuration store

MongoDB Atlas holds users, organizations, plantations and sensors. Plantation documents reference their sensors, and the login query populates that relationship so the dashboard can scope its views without additional round trips.

A document store was selected over a relational one because tenant structures differ between farms: attributes present on one operation's plantation are absent on another's. A document model absorbs that variation by adding fields, without a migration affecting every other tenant record. Atlas specifically because it is managed, which keeps database operations out of scope.

A purpose built time series store was considered and rejected, on the basis that it would consume development time reproducing capability that mature open source products already provide. InfluxDB was then selected over the alternatives on three grounds. Its data model fits an append only stream of timestamped device readings. It is available as a managed service, which removes an operational burden from a single developer. And its query language allows the threshold checks and scheduled tasks described in section 7 to run inside the database instead of in a separate processing tier, which is the property that most shaped the design. No benchmark was run against the alternatives, and no storage efficiency claim is made.

6.3 Validation, normalization and gap handling

Normalization uses mean aggregation over the query window, at intervals between five minutes and one hour depending on the series and the range requested.

Limitation. Mean is sensitive to a single anomalous sample, and sensor data produces anomalous samples. A median would be the more robust aggregate and is not what the query layer currently uses. No outlier rejection runs before aggregation either, which is the same gap section 5.7 records as the absence of range and rate of change validation.

Gaps are dropped rather than filled. Every aggregate window is built with `createEmpty: false`, so an interval in which a node reported nothing produces no row at all rather than an empty or carried forward one. The series returned to the dashboard is shorter, not flat.

Limitation. Nothing in the data path distinguishes a gap from a period that was not requested, and nothing raises an alert when one appears. A node that stops reporting therefore produces a series that simply ends, with no marker and no notification, and whether that reads as a fault depends on the person looking at the chart. Explicit gap marking, and the silence detection described in section 5.7, are both open.

6.4 Application services

Service	Stack	Function
API	Node.js, Express	Authentication, tenant scoped Flux queries, sensor and plantation CRUD, notification and check management
Forecasting	Python, Flask	Time series forecasting, reads from InfluxDB, writes results to InfluxDB and MongoDB
Frontend	React	Dashboard, visualization, configuration

6.5 Operating cost

The development instance runs on a workstation, on the free tiers of InfluxDB Cloud and MongoDB Atlas, so its running cost is effectively zero.

A deployment that gateways on farms can reach from anywhere needs a host with a fixed public address. At the published price of a one gigabyte Amazon Lightsail instance, which includes its public address, storage and transfer, that host costs about USD 7 a month, and the storage services stay on their free tiers at this scale. A host on the farm itself avoids that charge and adds the farm's own internet connection as a dependency.

At approximately 50 nodes on minimum paid tiers, the estimate is roughly USD 50 a month for compute and about USD 50 prorated across the managed databases, for a total of USD 100 to 150. The figure is derived from published tier pricing and not from a bill at that scale.

The unit is the deployment, not the farm. Section 5.5 records tenant isolation enforced server side, so one deployment of the stack serves more than one farm at the application layer, with the qualification at the network server recorded in section 8.2, and this cost is divided among the farms it serves rather than repeated for each.

7. Analytics and decision support

7.1 Threshold monitoring and alerting. Partially implemented.

A check named `humedad_check` evaluates soil moisture every fifteen minutes against a configured threshold and writes a status into the `_monitoring` bucket at level ok, warn or crit.

A scheduled task named `Humedad` runs hourly with a one minute offset, reads those statuses, assembles the list of devices in critical state, and sends an email through SendGrid via an HTTP POST issued from inside the database engine, using an API key held in the InfluxDB secret store. Notification records are written to `agro.notifications`.

Locating threshold evaluation and dispatch inside the time series engine removes the need for a separate polling service.

Limitation. In the development instance the check and the task are built and switched off. They evaluate readings against a threshold, so neither can detect a node that has stopped reporting; a silent node produces no reading to evaluate. Silence detection is planned work, listed in section 5.9.

7.2 Forecasting. Partially implemented.

A Flask service performs univariate time series forecasting with **Prophet**, the open source additive regression model from Meta. It retrieves a device history from InfluxDB, fits a model to that series, and writes the forecast back to InfluxDB and to MongoDB.

Two qualifications apply. Prophet is not a pretrained model; it fits parameters to the series supplied, decomposing it into trend and seasonal components. The deployed configuration is per series and univariate: it projects one sensor metric forward from that metric own history, without weather forecasts, irrigation events or crop stage as inputs.

Limitation. The service produces a trend projection, not an agronomic prediction. A multivariate model conditioned on those additional inputs is planned and not implemented.

7.3 Dashboard. Implemented.

The React dashboard presents per plantation soil moisture, soil temperature, battery level and NPK series, with selectable time windows, a table view of raw values, and node status.

7.4 Growth stage irrigation thresholds. Not validated.

A table of growth stage specific soil moisture thresholds was drafted during development.

Limitation. The table has not been validated against agricultural extension literature and no source is claimed for it. It operates as a working hypothesis. Establishing defensible thresholds requires either verification against published land grant extension guidance for the target crops, or replacement with values drawn from that literature. Until then, any threshold based recommendation is provisional.

7.5 Irrigation recommendation. Not implemented.

There is no irrigation recommendation engine. The system reports conditions, and its threshold alerting is built and switched off in the development instance (section 7.1). It does not issue an irrigation instruction.

The intended approach is to accumulate labeled data, soil moisture series alongside recorded irrigation events and outcomes, and to train a supervised model recommending timing and volume by crop. This depends on the season length history that section 6.1 records as unavailable, and on the validated thresholds that section 7.4 records as absent. Both are prerequisites and neither is met.

8. Cost of the architecture

The figures below are the cost of building and running the architecture. They are stated here because affordability is a design constraint, recorded in section 1. Pricing, margin and revenue are commercial matters and are not addressed in this document. Where this section cites prices published by other parties, it does so as a benchmark for that design constraint.

8.1 Bill of materials

One node configuration has been built and operated, and it is the one costed here. Six nodes exist. Two of them carried the sensors in the validation run of section 11; section 4.3 records four registered on the gateway.

Field ready node. USD 231.11. Every line is a component purchased at the price shown. The enclosure and glands are rated IP65 as supplied, so this is the cost of a node that can be left on a mast in a field.

Component	USD
LILYGO T-Beam V1.2 board: ESP32, LoRa radio, GPS, LiFePO4 cell and charging circuit	33.66
JXBS-3001 soil probe: moisture, temperature, EC, pH and NPK, Modbus RTU over RS485, 5 V DC	145.00
Pololu S9V11F5S6C step up / step down regulator with adjustable low voltage cutoff	8.50
MT3608 boost converter, DC-DC step up, on the probe supply	1.20
TTL to RS485 adapter module	4.50
Soshine 6 V 6 W solar panel	11.00
Zulkit IP65 outdoor project box	4.25
Cables, IP65 cable glands and miscellaneous materials	3.00
Mast and mounting elements	20.00
Total	231.11

The board price covers the LiFePO4 cell and the charging circuit, which is why no cell appears as a line of its own. The probe is 63 percent of the node, which follows from the selection recorded in section 3.1. The five dollar capacitive sensors tried first corroded at the soil contact points within two to three weeks, and at an unattended node a short lived part is a recurring cost rather than a cheap one. Service life governed the choice, and this table is what that decision cost.

A moisture and temperature instrument of the same industrial class, without the nutrient, pH and conductivity channels, is available for less and would reduce the node accordingly. Section 9 records that the nutrient channel does not calibrate and that no function depends on it, so the

decision layer would lose nothing. I have not bought one, and I state no figure for a configuration I have not built.

Gateway: RAK7249 at USD 469.00, including the antenna and the internal backup battery. One time, shared across every node within the range recorded in section 4.3.

Installation: approximately one day of work per farm, costed at USD 300.

8.2 How cost scales

The node is the only component whose cost scales with the area instrumented. Everything else is fixed, at two different levels.

Fixed per farm. The gateway at USD 469 once, antenna and backup battery included, and the internet link at the gateway at about USD 30 a month, or USD 360 a year. One gateway covers every node within its range.

Fixed per deployment, not per farm. The cloud services cost between USD 100 and USD 150 a month at the minimum paid service tiers, and the figures below use the lower end of that range, which is where the component estimates in section 6.5 sum. At USD 1,200 a year, the deployment carries this cost once: section 5.5 records that the application enforces tenant scoping on every query, so one deployment serves more than one farm at the application layer. Section 5.9 records that all devices currently sit under a single tenant at the network server and that per farm separation there is not yet configured, so the sharing is a property of the application and not yet of the whole stack.

At one node per acre, equipment capital per acre is the node cost of 231 plus 469 divided by the acreage:

Acres under one gateway	Capital per acre, USD
10	278
50	240
100	236
500	232

The table excludes the installation allowance. Adding it raises the ten acre figure by 30 dollars an acre and the hundred acre figure by 3.

One node per acre is an engineering judgment, not a validated density. It is the density the architecture assumes for a first season, and no first season has been run. Every per acre figure in this section inherits that assumption.

On soils heterogeneous enough to require it, sensing density rises to two nodes per acre. That determination waits for a first season at one node per acre, because it depends on the variability that season measures. Doubling density doubles the node cost per acre and adds nothing recurring.

Limitation. The cloud figure is derived from published service tier pricing, not from a bill at operating scale. Section 6.5 records the same qualification.

8.3 Benchmark against commercially available systems

This subsection tests the design constraint in section 1 against what the market charges. It is a benchmark for an engineering target. No part of this architecture is offered as cheaper than any named product.

At the level of the individual device, this architecture holds no cost advantage.

Integrated LoRaWAN soil sensors sell for well under what a node costs to build here: the Seeed SenseCAP S2104 lists at USD 149.99 from the manufacturer and the Dragino LSE01 at about USD 145.78 from a United States distributor, against USD 231.11 for the node in 8.1. Both are complete sealed units rather than partial components, and neither includes a gateway, a network server or an application layer.

Most of that difference is the probe. The JXBS-3001 is 145 dollars of a 231 dollar node, and it reads five quantities the SenseCAP does not. The rest follows from the design choice recorded in section 3.2: both products run on primary, non rechargeable lithium thionyl chloride cells and neither offers a solar option, while this architecture charges a rechargeable cell from a panel, which puts the panel, the regulator and the mast on the bill of materials. Solar into a rechargeable cell is a different model of unattended operation, and it carries that cost.

A node on its own does not produce an irrigation decision, so the individual device is the wrong level at which to test the design constraint in section 1.

At the level of a complete system there is a material difference. The comparison below uses Onset HOBOnet, whose prices are published on the manufacturer's own site, for a ten acre deployment at ten sensing points, at the field ready node cost from 8.1:

	Capital	Recurring
Onset HOBOnet, moisture only sensors	4,858: RX3000 station 1,079, HOBOnet wireless manager 229, ten RXW-SMD-900 at 355	199 a year, SP-811 plan
Onset HOBOnet, moisture, temperature and EC sensors	7,458: same station and manager, ten RXW-T12 at 615	199 a year, SP-811 plan
This architecture, 231 dollar field ready node	2,780: ten nodes at 231.11, gateway 469	360 a year for the farm link, plus a share of the 1,200 a year stack

Onset's data plan is mandatory. The RX3000 page states that the station "Requires an RX data plan, which must be renewed annually." The plan is priced per station and capped by sensor count and connection rate: SP-811 at USD 199 covers 20 sensors at hourly connections and SP-813 at USD 299 covers 25 sensors at ten minute connections, with a 65 sensor premium tier available by quotation only. The plan cap binds before the hardware does: one wireless manager supports up to 50 sensors, but a deployment above 20 or 25 sensors already requires a higher plan tier, and above 65 it requires a second station with a plan of its own.

Two qualifications on the recurring figures, both against the comparison as drawn:

First, the annual fees are not equivalent in what they buy. Onset's plan covers cellular telemetry and a hosted data portal with dashboards, alarms and scheduled export to CSV or XLSX, with an API limited to 30 requests per minute and ten loggers per query. It includes no compute, no queryable database, no application hosting and no multi tenancy. The 1,200 dollar figure in this document covers compute, a time series store, a document store, application hosting and tenant isolation, on infrastructure under the operator's control. The two numbers are not the same kind of cost.

Second, the market is not uniform on interoperability, and it would be inaccurate to describe it as though it were. Onset's HOBOnet is a proprietary 900 MHz network whose only telemetry path is the vendor's platform, with no self hosting option. Decentlab, by contrast, publishes LoRa Alliance certified devices with open payload codecs that run on any LoRaWAN network server, so its hosted service is optional. The interoperability barrier that GAO-24-105962 reported to Congress is real and present in this market, and it is present in some products and not others.

Sources for this subsection, all accessed September 2026:

- Onset Computer Corporation, RX3000 Remote Monitoring Station, HOBOnet Wireless Manager, and RX data plan product pages.
<https://www.onsetcomp.com/products/data-loggers/rx3000> and
<https://www.onsetcomp.com/products/communications/rxmod-rxw-xxx>
- Seeed Studio, SenseCAP S2104 product page and SenseCAP S210X LoRaWAN Sensor User Guide. <https://www.seeedstudio.com/SenseCAP-S2104-LoRaWAN-Soil-Temperature-and-Moisture-Sensor-p-5357.html>
- Dragino, LSE01 LoRaWAN Soil Moisture and EC Sensor user manual and product page. <https://www.dragino.com/products/lora-lorawan-end-node/item/159-lse01.html>
- Decentlab, Services. <https://www.decentlab.com/services>
- Elshikha, D. E., et al., "Guidance for Soil Moisture Sensor Selection: Market Analysis and Decision-Making Strategies," University of Arizona Cooperative Extension, publication az2082, March 2024, consulted for the structure of the market and the separation of equipment cost from data plan cost.
<https://extension.arizona.edu/publication/guidance-soil-moisture-sensor-selection-market-analysis-and-decision-making-strategies>

8.4 Cost control measures

Direct purchase from the sensor manufacturer rather than through resellers.

Integrated board rather than separate modules. The T-Beam carries ESP32, GPS, LoRa radio, battery and charging circuit on one board. The equivalent discrete modules cost more and added integration work.

Standardization on ESP32, for firmware stability and library availability, which reduced development time.

Unlicensed radio band rather than cellular, which removes a per device subscription from the recurring cost and removes the per station plan requirement that the Onset system surveyed in section 8.3 carries.

9. Register of open items and limitations

Each entry is cross referenced to the section where it is described. Taken together, these entries define the engineering work that remains before the architecture is complete, and they set the order in which it is being done.

NPK measurement is not reliable and is not used. Temperature and moisture calibrated cleanly against a reference in a single calibration event and have remained stable since, which section 3.4 qualifies. NPK did not calibrate at all. Validation requires laboratory soil analysis for comparison, and in internal testing two units of the same 7 in 1 probe reported different values for the same soil. Only temperature and moisture are treated as usable. NPK, pH and EC are read and stored, and no function depends on them. A separate NPK sensing approach using multispectral soil imaging with machine learning interpretation, built from off the shelf ESP32 and Arduino components, is under consideration. It is exploratory, no prototype exists, and the component cost of about USD 150 is a parts list total. Two obstacles stand in the way and neither has a designed solution. It requires a trained model to interpret the images, which does not exist and for which no labeled dataset has been assembled. It also requires a way to move image data over a LoRaWAN link, and LoRaWAN uplinks carry payloads of tens to a few hundred bytes, so images cannot be transmitted over this radio layer without on device processing that reduces them to features first. Section 3.4.

Battery autonomy is three days at a one minute transmission interval in good sun. The autonomy against interval relationship has not been characterized, and performance under extended overcast is not characterized. No alarm is raised when a node goes silent, so a depleted battery is not visible until the dashboard is inspected. Between 23 and 30 September 2026 the deployed nodes ran down their batteries and stopped reporting, and nothing flagged it for a week. Sections 3.2 and 11.

The enclosure rating is the manufacturer's, not an independent test. The housing and glands on the six built nodes are rated IP65 as supplied, which covers dust and water jets from any direction. I have not tested an assembled node with its penetrations made, and no node has completed a full growing season in the field. Section 3.3.

The frame format carries defects that the published specification records. Appendix A documents the layout completely, and with it five faults. Mask bit 5 is overloaded: it signals both that a rain sensor is present and that the frame is a full seven parameter frame, so a node cannot report one without asserting the other. The decoder validates no length, so a truncated uplink yields arithmetic on undefined bytes and writes NaN. The pH and conductivity fields are stored as the raw sixteen bit values with no scaling applied, so neither is in its physical unit. Every byte of the 42 byte frame is allocated, so a further sensor class requires extending the frame, even though bit 0 of the mask is unused. And the frame carries no version field, so a receiver cannot tell which revision of the format it is decoding. Section 4.2 and Appendix A.

The storage and identity layers depend on managed services. InfluxDB Cloud and MongoDB Atlas were chosen to remove operational burden from a single developer. Both are proprietary managed services, so a different builder would have to substitute equivalents, and the reference documentation has to state what the substitution requires. Sections 6.1 and 6.2.

No calibration schedule exists. Temperature and moisture were calibrated in a single event, and stability since then is an observation, not the result of periodic comparison against a reference. Section 3.4.

The operating cost figure is an estimate from published pricing, not a bill. No invoice at operating scale exists. Section 6.5 and section 8.2.

Node ceiling per gateway is not characterized. Section 4.3.

The deregistration fault is unresolved. Nodes intermittently lose registration and stop reporting without raising a fault. A reset every fifty uplinks masks it without correcting it, and its cause has not been confirmed. Section 4.5.

The AppKey is stored unencrypted in node flash, so physical access to a node exposes it. ESP32 flash encryption would close this; it is irreversible on the hardware, so it is deferred to the next hardware build. Section 5.12.

Firmware updates are unsigned and require physical access. Section 5.12.

The deployed firmware does not check the length of a downlink. A downlink longer than the three byte command overruns the buffer it is copied into. Only the network server can send one. Version 31 corrects it and has not yet been loaded onto the nodes. Section 5.12.

Threshold alerting is switched off in the development instance, and it cannot detect a silent node. Section 7.1.

The development instance departs from the deployment requirements in four respects: the forecasting service runs on Python 3.9, which no longer receives security fixes; the gateway port is open to the whole local network; InfluxDB Cloud is reached by token alone, with no address restriction; and the credential files are readable by the workstation's user account. Section 5.11.

The gateway is identified by its EUI alone, over a protocol that carries no authentication. Section 5.3.

No role system beyond the organization hierarchy, and a single administrative account at the network server layer. Section 5.5.

No reading integrity checks. Section 5.7.

The node keeps no record of configuration changes and does not detect or report tampering. Section 5.12.

No tested recovery protocol. Section 5.8.

Internal PostgreSQL and Redis connections are unencrypted. Section 5.10.

Planned security work at the network server and the application is not done: per farm tenants and private gateway flags in ChirpStack, MQTT over TLS on the gateway bridge, tenant scoped InfluxDB tokens, shorter tokens with refresh and audit logging of access, and LoRaWAN 1.1. Section 5.9.

Retention is thirty days, shorter than a growing season. Section 6.1.

Gaps are dropped, not marked, and nothing alerts on them. A node that stops reporting produces a series that simply ends. Section 6.3.

Aggregation uses the mean, with no outlier rejection ahead of it. A single anomalous sample moves the result. Section 6.3.

Forecasting is univariate. It projects one metric forward from its own history, with no weather, irrigation events or crop stage as inputs. Section 7.2.

Growth stage thresholds are unvalidated. Section 7.4.

No irrigation recommendation engine. Section 7.5.

The gateway is a single point of failure. Partially mitigated: it carries a redundant battery and a field rated enclosure. Outright failure would require replacement, and a replacement within a couple of days would not break the continuity of a data series already collected. There is no hot standby. Section 4.3.

Range is degraded by obstruction. The 5 km figure is line of sight. Foliage and branches between node and gateway degrade the link, and the mitigation is mast mounting above the canopy. An area beyond one gateway's range needs a second gateway forwarding to the same network server, which this deployment has not tested. Sections 3.3 and 4.3.

10. Development history

Period	Activity
Late 2019	Started as an IoT proof of concept. Agriculture was chosen as the application domain because it was the one I could reach from where I was living, and because the problem is the same wherever irrigation draws on a finite aquifer
2019 to March 2020	Relearned web development, React and current JavaScript frameworks. First components purchased and tested with low cost sensors, which failed
2020 to 2021	Arduino IDE development during the pandemic, RS485 and Modbus study, then LoRaWAN research
Mid 2021	First working end to end prototype. First project website in vanilla JavaScript
2022	Continued testing. RAK Wireless hardware adopted as the deployment gateway, with the Raspberry Pi gateway still running alongside it. Initial conversations with prospective

Period	Activity
	farms about field testing
2023	More robust build. Raspberry Pi gateway disconnected. Site rebuilt on Hugo and React, with a demonstration view using Plotly
2024	Paused. Internal security conditions in Ecuador constrained travel to remote sites and slowed prospective partners
2025	Sensor modifications aimed at measurement efficiency
2026	Thirty one day end to end validation with the cloud infrastructure. Field validation agreement executed.

Development spanned seven years with intermittent activity. Most layers were built, tested against field conditions, and in several cases rebuilt after the initial approach failed in deployment.

11. Operational validation

The architecture was deployed and operated for thirty one days in a controlled growing environment, exercising the full path from sensor acquisition through LoRaWAN transmission, network server processing, cloud ingestion, storage and dashboard presentation.

The operating record is maintained separately. It shows the architecture running end to end under growing conditions and producing continuous data. The run took place in a covered greenhouse in Guayaquil, Ecuador, in potting containers, on one crop over one cycle, with two instrumented sensing points. It does not constitute a field trial at agricultural scale, it is not offered as evidence of agronomic outcomes, and validation on United States farms is work that remains ahead.

The run required no site visit, and one qualification belongs on that statement. The nodes carried the periodic reset described in section 4.5 throughout. That measure masks an intermittent deregistration fault without correcting it. Continuous operation over the thirty one days was achieved with that workaround in place.

Between 23 and 30 September 2026 the deployed nodes stopped reporting because their batteries had run down. Nothing raised an alert. The silence was found a week later by inspecting the network server, and reporting resumed the same day once the nodes were recharged. It is the failure that the silence detection in section 5.7 is meant to catch, and the register in section 9 carries it.

On 30 September 2026, after recharging, four nodes were reporting to an online gateway.

Appendix A. Uplink payload specification

This appendix documents the AgroIQ uplink frame completely enough that an independent implementer can write a decoder in any language without access to mine. It is published under the license stated at the front of this document.

A.1 Frame

A single uplink of **42 bytes**, big endian throughout, at fixed offsets. The frame length does not vary with the sensors fitted. Byte 17 carries a capability mask recording which sensors produced a reading; a field whose mask bit is clear still occupies its bytes in the frame, and is simply left out of the decoded record.

A.2 Fields decoded unconditionally

Bytes	Field	Type	Conversion	Unit
0–2	latitude	uint24	$(\text{raw} / 16777215 \times 180) - 90,6$ dp	degrees
3–5	longitude	uint24	$(\text{raw} / 16777215 \times 360) - 180,6$ dp	degrees
6–7	altitude	int16	bit 7 of byte 6 is the sign; if set, sign extend	meters
8	hdop	uint8	$\text{raw} / 10$	none
9	sats	uint8	as read	count
17	capab	uint8	as read	bit mask, see A.4
18–19	voltage	uint16	$\text{raw} / 1000$	volts

A.3 Fields gated by the capability mask

Mask condition	Bytes	Field	Conversion	Unit
bit 1	12–13	temp	$\text{raw} / 100$	°C
bit 1	14–15	hum	$\text{raw} / 100$	%
bit 2	10–11	soil	$((\text{raw} - 1200) \times -1/30) + 100,$ 2 dp	%
bit 3	16	relay	as read	state
bit 4	20–21	temp2	$\text{raw} / 100$	°C
bit 4	22–23	hum2	$\text{raw} / 100$	%
bit 4	24–26	press	uint24, as read	none
bit 5	27	rain	as read	none

Mask condition	Bytes	Field	Conversion	Unit
bit 6 set and bit 7 clear	28–29 / 30–31 / 32–33	N / P / K	as read	mg/kg
bit 7 set and bit 6 clear	34–35	ph	as read, unscaled	raw
bit 7 set and bit 6 set	36–37	ec	as read, unscaled	raw
bits 5, 6 and 7 all set	28–33	N / P / K	as read	mg/kg
bits 5, 6 and 7 all set	34–35	ph	as read, unscaled	raw
bits 5, 6 and 7 all set	36–37	ec	as read, unscaled	raw
bits 5, 6 and 7 all set	38–39	soil_temp	raw / 10, 2 dp	°C
bits 5, 6 and 7 all set	40–41	soil_moist	(raw / 10) – 40	% VWC

The `soil` field at bytes 10–11 is the legacy channel for the capacitive sensor described in section 3.1, the one that corroded. It is retained in the layout for frames from earlier builds and no current node populates it.

The `soil_temp` and `soil_moist` conversions carry the offsets applied after the calibration described in section 3.4: a divide by ten on temperature and a divide by ten with a minus forty offset on moisture.

A.4 Capability mask, byte 17

Bit	Meaning
0	unused
1	air temperature and humidity present
2	legacy capacitive soil channel present
3	relay state present
4	barometric sensor present: temperature, humidity and pressure
5	rain sensor present, and part of the seven parameter selector
6	nutrient channels present, and part of the seven parameter selector
7	pH and conductivity selector, and part of the seven parameter selector

The seven parameter probe described in section 3.1 is signaled by bits 5, 6 and 7 set together.

A.5 Known defects in this format

The format grew a sensor at a time, and these are the consequences. An implementer needs them, and section 9 carries them as open items.

Bit 5 is overloaded. It signals both that a rain sensor is fitted and that the frame is a full seven parameter frame. A node cannot report one condition without asserting the other, so a seven parameter node necessarily claims a rain sensor and byte 27 is decoded as a rain reading whether or not one exists.

No length validation. The format defines no minimum length and my decoder checks none. A truncated uplink produces arithmetic on undefined bytes and yields NaN, and nothing downstream distinguishes that from a reading. Section 5.7 records the same gap as the absence of range and rate of change validation.

pH and conductivity are unscaled. Both are stored as the raw sixteen bit values. Neither is in a physical unit, and the scale factor the probe applies is not documented by its manufacturer in a form I have been able to verify. Section 9 records that these channels are not relied on by any function, which is why the omission has had no operational consequence, and it is a defect in the format regardless.

The format has no room to grow. Bit 0 of the mask is unused, but every byte of the 42 byte frame is allocated, three mask bits carry combined meanings, and a further sensor class cannot be added without extending the frame.

No format version field. The frame carries no version identifier, so a receiver cannot tell which revision of this specification a given uplink was produced under. The `soil_temp` and `soil_moist` offsets in A.3 changed during development, and a frame from before that change decodes to the wrong values with no indication.

A.6 Worked example

A seven parameter node with bits 5, 6 and 7 set reports `capab` = 224 plus whichever of bits 1 to 4 apply. Given bytes 38–39 = 0x00F5 and bytes 40–41 = 0x02A6, the node reports a soil temperature of 24.50 °C and a volumetric water content of 27.8 percent.